

Solutions to exercises in Lecture 3

Q1) The solution converges fastest when using the optimal ω . For larger ω , the solution overshoots, which leads to slower convergence.

For smaller ω , especially when $\omega < 1$, the convergence is slow. In this problem, the initial condition is quite far from the solution. Hence, putting a large weighting to the previous iteration is disadvantageous.

Q2) We can write:

$$\epsilon_{12} = \frac{U(AX_2) - U(AX_1)}{U(AX_1)} = \frac{C[(AX_2)^p - (AX_1)^p]}{U(AX_1)}$$

$$\begin{aligned} GCT_{12} &= \frac{F_S C [(AX_2)^p - (AX_1)^p]}{U(AX_1) \left[\left(\frac{AX_2}{AX_1} \right)^p - 1 \right]} = \frac{F_S C (AX_1)^p \left[\left(\frac{AX_2}{AX_1} \right)^p - 1 \right]}{U(AX_1) \left[\left(\frac{AX_2}{AX_1} \right)^p - 1 \right]} \\ &= \frac{F_S C (AX_1)^p}{U(AX_1)} \end{aligned}$$

Similarly

$$GCT_{23} = \frac{F_S C (AX_2)^p}{U(AX_2)}$$

$$\text{Thus: } \frac{GCT_{23}}{r^p GCT_{12}} = \frac{\frac{F_S C (AX_2)^p}{U(AX_2)}}{\left(\frac{AX_2}{AX_1} \right)^p \frac{F_S C (AX_1)^p}{U(AX_1)}} = \frac{U(AX_1)}{U(AX_2)} \approx 1.$$

Q3) SOR is the most effective algorithm as it needs the fewest iterations to converge. However Jacobi can be implemented most efficiently using Python. In terms of performance, Jacobi iteration harnessing Python's array operation performs best.